



COMEÇAMOS EM BREVE!

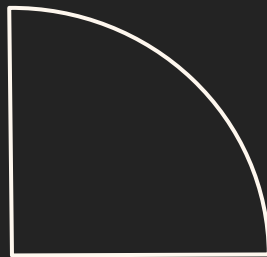
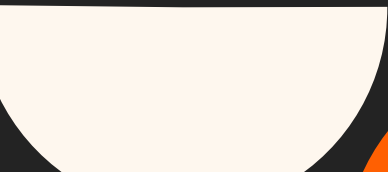
Iniciaremos a aula às 08h10.





Engenharia de Dados

2026 | MARÇO



Apache Spark

O Apache Spark é um sistema de **processamento distribuído** de código aberto para big data. Utiliza armazenamento em cache na memória e execução otimizada para consultas analíticas rápidas em dados de qualquer tamanho.





Oferece APIs em Java, Scala, Python e R, permitindo a reutilização de código em várias workloads, como processamento de lotes, consultas interativas, análises em tempo real, machine learning e processamento de gráficos.



Origem e Início:

- Iniciado em 2009 como um projeto de pesquisa no AMPLab da UC Berkeley.
- Colaboração entre estudantes, pesquisadores e professores.



Objetivo:

- Criar uma estrutura otimizada para processamento iterativo rápido, como machine learning e análise interativa de dados.
- Escalabilidade e a tolerância a falhas.



Primeiro Artigo:

- "Spark: Cluster Computing with Working Sets" publicado em junho de 2010.
- Spark lançado como código aberto sob licença BSD.



Transição para Apache:

- Tornou-se um projeto de incubação na Apache Software Foundation (ASF) em junho de 2013.
- Estabelecido como um projeto de alto nível da Apache em fevereiro de 2014.

Funcionamento do Spark

Antes de 2009, a principal ferramenta de processamento de dados era o Hadoop MapReduce, um modelo de programação projetado para big data que utiliza algoritmos distribuídos e paralelos. Ele permite que desenvolvedores criem operadores altamente paralelizados sem se preocuparem com a distribuição do trabalho e a tolerância a falhas.





No entanto, um desafio era o processo sequencial de várias etapas necessário para executar um trabalho. Pois, em cada etapa, o MapReduce lê dados do cluster, executa operações e grava os resultados no HDFS. Isso resulta em lentidão devido à latência da E/S do disco, pois cada etapa exige leitura e gravação no disco.



O Spark foi criado para resolver as limitações do MapReduce, processando dados na memória e reduzindo etapas de tarefas. Ele executa operações em uma única etapa, o que acelera a execução.



Utiliza cache na memória para reutilizar dados em várias operações, especialmente em algoritmos de machine learning, através de DataFrames e RDDs (Resilient Distributed Datasets). Isso reduz drasticamente a latência, tornando o Spark várias vezes mais rápido que o MapReduce.



Embora o design do Spark e do Hadoop MapReduce seja diferente, muitas organizações os utilizam juntos para resolver desafios comerciais mais amplos.

O Spark é uma estrutura de código aberto voltada para consultas interativas, machine learning e workloads em tempo real. Ele executa análises em sistemas de armazenamento como HDFS, Amazon Redshift, Amazon S3, Couchbase e Cassandra.



No Hadoop, o Spark usa o YARN para compartilhar clusters e conjuntos de dados comuns com outros mecanismos do Hadoop, garantindo serviços e respostas consistentes.



PySpark é uma API em Python para executar o Spark e foi lançado para oferecer suporte à colaboração entre Apache Spark e Python. O PySpark também oferece suporte à interface do Apache Spark com conjuntos de dados distribuídos resilientes (RDDs) na linguagem de programação Python.

Benefícios do Spark



Rápido

Por meio do armazenamento em cache na memória e execução otimizada de consultas, o Spark pode oferecer consultas analíticas rápidas de dados de qualquer tamanho.



Para desenvolvedores

O Apache Spark sustenta de modo nativo Java, Scala, R e Python, oferecendo a você várias linguagens para a criação de aplicativos. Essas APIs facilitam as coisas para seus desenvolvedores, pois ocultam a complexidade do processamento distribuído por trás de operadores simples e de alto nível que reduzem drasticamente a quantidade de código necessária.



Várias workloads

O Apache Spark vem com a capacidade de executar várias workloads, incluindo consultas interativas, análises em tempo real, machine learning e processamento de grafos. Uma aplicação pode combinar várias workloads facilmente.

*Workloads do
Apache Spark*



A estrutura do Spark inclui:

- **Spark Core** como base para a plataforma
- **Spark SQL** para consultas interativas
- **Spark Streaming** para análises em tempo real
- **Spark MLlib** para machine learning
- **Spark GraphX** para processamento de grafos

MLlib

Machine Learning

Streaming

Real-time analytics

SQL

Interactive Queries

GraphX

Graph processing

APACHE
Spark Core

R

Python

Scala

Java



Spark Core

O Spark Core é a base da plataforma, gerenciando memória, recuperação de falhas, programação, distribuição e monitoramento de tarefas, além de interagir com sistemas de armazenamento. Ele oferece APIs para Java, Scala, Python e R, simplificando o processamento distribuído com operadores de alto nível.



MLlib

O Spark inclui o MLlib, uma biblioteca de algoritmos de machine learning para dados em grande escala. Cientistas de dados podem treinar modelos com R ou Python, salvar com MLlib e importar para pipelines em Java ou Scala.



Projetado para computação rápida e interativa na memória, o Spark permite execução rápida de machine learning. Os algoritmos disponíveis incluem classificação, regressão, agrupamento, filtragem colaborativa e mineração de padrões.



Spark Streaming

Solução em tempo real que utiliza o agendamento rápido do Spark Core para análises de streaming. Ele processa dados em minilotes, permitindo o uso do mesmo código de aplicação para análises em lotes e streaming, aumentando a produtividade do desenvolvedor.



O Spark Streaming é compatível com dados de Twitter, Kafka, Flume, HDFS, ZeroMQ, entre outros do ecossistema Spark.



Spark SQL

É um mecanismo de consulta distribuído que oferece consultas interativas de baixa latência, até 100 vezes mais rápidas que o MapReduce. Inclui um otimizador baseado em custos, armazenamento colunar e geração de código para consultas rápidas e escaláveis em milhares de nós.



Permite que analistas usem SQL padrão ou Hive Query Language e desenvolvedores utilizem APIs em Scala, Java, Python e R. Suporta várias fontes de dados, como JDBC, ODBC, JSON, HDFS, Hive, ORC, Parquet, Amazon Redshift, Amazon S3, Couchbase, Cassandra, MongoDB, Salesforce.com e Elasticsearch.



Spark GraphX

O Spark GraphX é uma estrutura distribuída de processamento de grafos construída sobre o Spark. Ela oferece ETL, análise exploratória e computação gráfica iterativa, permitindo a criação e transformação interativa de grandes estruturas de dados gráficos. O GraphX inclui uma API flexível e uma seleção de algoritmos gráficos distribuídos.



Mais detalhes:

<https://aws.amazon.com/pt/what-is/apache-spark/>

0 PySpark



Para rodar o PySpark local é necessário usar o Docker ou ter o Java no seu SO, para termos a nossa primeira experiência com esse importante framework vamos usar a versão do Google Colab.



Cada bloco de código a seguir estará em uma célula do Colab

```
!apt-get install openjdk-11-jdk-headless -qq > /dev/null
```

```
!pip install -q pyspark
```



```
from pyspark.sql import SparkSession
```

```
spark =
```

```
SparkSession.builder.appName ("Spark-ProcessamentoMeiosPagamento" ).getOrCreate ()
```

```
spark
```

SparkSession - in-memory

SparkContext

[Spark UI](#)

Version

v4.0.2

Master

local[*]

AppName

Spark-ETL



Vamos usar a API do banco central na aula de de hoje:

[https://olinda.bcb.gov.br/olinda/servico/MPV_DadosAbertos/versao/v1/s
wagger-ui3#/](https://olinda.bcb.gov.br/olinda/servico/MPV_DadosAbertos/versao/v1/s
wagger-ui3#/)

```
import requests

def extract meios pagamentos(trimestre: str, n retornos: str):
    """
    Extraí dados abertos sobre meios de pagamento trimestrais da API do Banco Central do Brasil.

    Args:
        trimestre (str): O trimestre e ano desejado no formato 'YYYYQ' (ex: '20251' para o 1º
        trimestre de 2025).
        n retornos (str): O limite de registros que a API deve retornar (cláusula $top).

    Returns:
        dict: Um dicionário onde a chave 'value' contém a lista com os dados financeiros extraídos.
    """
    url =
f"https://olinda.bcb.gov.br/olinda/servico/MPV_DadosAbertos/versao/v1/odata/MeiosdePagamentosTrim
estralDA(trimestre=@trimestre)?@trimestre='{trimestre}'&$top={n retornos}&$format=json&$select=da
tatrimestre,valorPix,valorCartaoCredito,valorCartaoDebito,quantidadePix,quantidadeCartaoCredito,q
uantidadeCartaoDebito"

    response = requests.get(url)
    response.raise_for_status()
    meios pagamentos = response.json()

    return meios pagamentos
```



```
meios pag = extract meios pagamentos(trimestre="20191", n retornos=100)  
dados = meios pag.get("value", [])
```




```
from pyspark.sql.functions import col, to_date, round
```

```
# Criar DataFrame a partir dos dados da API
```

```
df = spark.createDataFrame (dados)
```

```
# Transformações de Data e Limpeza
```

```
# Converter a string 'datatrimestre' para o tipo Date real do Spark
```

```
df transformado = df.withColumn ("data_trimestre", to_date (col ("datatrimestre"),  
"yyyy-MM-dd")) \  
    .drop ("datatrimestre")
```



```
# Ajustar as escalas (Valores estão em Milhões, Quantidades em Milhares)
colunas_valor = [c for c in df.columns if c.startswith('valor')]
colunas_qtd = [c for c in df.columns if c.startswith('quantidade')]
```



```
from pyspark.sql.types import DoubleType
```

```
for c in colunas valor:
```

```
    # Transforma o valor na unidade absoluta (Multiplica por 1 Milhão)
```

```
    df transformado = df transformado.withColumn(c, round(col(c).cast(DoubleType()) * 1000000, 2))
```



```
for c in colunas_qtd:
```

```
    # Transforma a quantidade na unidade absoluta (Multiplica por 1 Mil)
```

```
    df_transformado = df_transformado.withColumn(c, round(col(c).cast(DoubleType()) * 1000, 2))
```



```
print("\n--- Schema do DataFrame Processado ---")
df.transformado.printSchema()
```

```
--- Schema do DataFrame Processado ---
root
 |-- quantidadeCartaoCredito: double (nullable = true)
 |-- quantidadeCartaoDebito: double (nullable = true)
 |-- quantidadePix: double (nullable = true)
 |-- valorCartaoCredito: double (nullable = true)
 |-- valorCartaoDebito: double (nullable = true)
 |-- valorPix: double (nullable = true)
 |-- data_trimestre: date (nullable = true)
```

```
print("\n--- Amostra dos Dados Processados ---" )  
df_transformado.show (truncate=False)
```

--- Amostra dos Dados Processados ---

quantidadeCartaoCredito	quantidadeCartaoDebito	quantidadePix	valorCartaoCredito	valorCartaoDebito	valorPix	data_trimestre
5.86075226E9	4.3035735E9	2.228725807E10	8.2478891E11	2.5382031E11	1.021586033E13	2025-12-31
5.58866917E9	4.08637358E9	2.048509912E10	7.5771144E11	2.3573158E11	9.27381425E12	2025-09-30
5.39842043E9	4.03017537E9	1.935715972E10	7.2709173E11	2.3637391E11	8.36373584E12	2025-06-30
5.16819664E9	3.97238423E9	1.751276247E10	6.912888E11	2.3360896E11	7.47106069E12	2025-03-31
5.35923634E9	4.28270654E9	1.793814661E10	7.291624E11	2.5994783E11	7.71427676E12	2024-12-31
5.04492785E9	4.12204559E9	1.65477431E10	6.634834E11	2.4194801E11	6.97833892E12	2024-09-30
4.88168051E9	4.05814352E9	1.535688734E10	6.3187694E11	2.3911432E11	6.2842943E12	2024-06-30
4.72553747E9	3.95251651E9	1.357545512E10	6.0763037E11	2.3512207E11	5.42930518E12	2024-03-31
4.81496527E9	4.23879424E9	1.304566989E10	6.3009774E11	2.5906831E11	5.300155E12	2023-12-31
4.58477143E9	4.01447199E9	1.111694786E10	5.8188206E11	2.4083587E11	4.46952957E12	2023-09-30
4.43515566E9	3.92986709E9	9.39657883E9	5.6173224E11	2.3817801E11	3.90037722E12	2023-06-30
4.18916915E9	3.83524667E9	8.11639331E9	5.3539502E11	2.3717906E11	3.44719557E12	2023-03-31
4.33326759E9	4.11547538E9	7.80716493E9	5.6027432E11	2.6412428E11	3.34291367E12	2022-12-31
4.14998637E9	3.83107959E9	6.52860208E9	5.2147932E11	2.436329E11	2.93641395E12	2022-09-30
3.97177641E9	3.77854037E9	5.44744822E9	5.0083677E11	2.4556375E11	2.5433844E12	2022-06-30
3.67738406E9	3.58996833E9	4.25002375E9	4.5280472E11	2.3018687E11	2.06782655E12	2022-03-31
3.73788365E9	3.84920818E9	3.8362856E9	4.6703792E11	2.574583E11	1.91641868E12	2021-12-31
3.3088641E9	3.52606288E9	2.86333694E9	3.9844024E11	2.3397003E11	1.55691622E12	2021-09-30
2.83931536E9	3.14169141E9	1.87156389E9	3.4433959E11	2.1438106E11	1.10573521E12	2021-06-30
2.58236334E9	3.00500465E9	8.6233786E8	3.0999959E11	2.0475291E11	6.2504652E11	2021-03-31

only showing top 20 rows

```
import sqlite3
```

```
def save_spark_to_sqlite_local(df, db_name, table_name, mode="append"):
```

```
    """
```

```
    Salva um DataFrame do PySpark em um banco SQLite local convertendo para Pandas,  
    evitando a necessidade de drivers JDBC e problemas de tipos de dados nativos.
```

```
    Args:
```

```
        df (DataFrame): O DataFrame do PySpark a ser salvo.
```

```
        db_name (str): Nome do arquivo de banco de dados (sem o .db).
```

```
        table_name (str): Nome da tabela.
```

```
        mode (str): Comportamento de escrita ('append' ou 'replace').
```

```
    """
```

```
    pandas_mode = "replace" if mode == "overwrite" else mode
```

```
    pdf = df.toPandas()
```

```
    con = sqlite3.connect(f"{db_name}.db")
```

```
    pdf.to_sql(table_name, con, if_exists=pandas_mode, index=False)
```

```
    con.close()
```

```
    print(f"Dados processados pelo Spark salvos com sucesso na tabela '{table_name}'!")
```



```
save spark to sqlite local(  
    df=df transformado,  
    db name="meios_pagamento",  
    table name="meios_pagamento trimestral"  
)
```


*Para quem gosta de
ir além:*

Deixei uma versão com o docker configurado para rodar localmente no repositório:

https://github.com/MatmJr/ETL_UNI/tree/aula_spark

Atividade



Ao longo das últimas aulas, aprendemos a consumir dados de uma API e armazená-los em um banco de dados MongoDB Atlas. Também estudamos como orquestrar pipelines de dados utilizando o Prefect.

Agora, o desafio será avançar para a etapa de transformação dos dados. Para isso, vocês deverão utilizar o PySpark para acessar os dados armazenados no MongoDB Atlas, processá-los e convertê-los em dados estruturados, organizados em formato tabular, adequados para análise e futuras etapas de visualização ou armazenamento.



Marco Mialaret



matmj@cesar.school



[linkedin](#)

